Python exam cheat sheet

ITS 303 • Quick reference based on your supplied six-domain summary. Compact additions are marked Supplement. Check your exam rules before bringing reference material.

1 Types and conversion

Python is dynamically typed: names refer to objects; objects have types. Names are case-sensitive. Use snake_case for variables and functions. Names cannot start with a digit or be a Python keyword.

Type	Example or conversion	Remember
int	`int("12")  # 12` `int(-3.9)  # -3`	Whole numbers; int(float) truncates towards zero.
float	`float("3.5")  # 3.5` `float(5)  # 5.0`	Binary floating point; many decimal fractions are approximate.
str	`str(5)  # "5"` `"5" * 2  # "55"`	Immutable text; use matching single or double quotes.
bool	`bool(0)  # False` `bool("False") # True`	False values include 0, None and empty strings or collections.
Conversion traps	`int("3.5")  # ValueError` `"3" + 2  # TypeError`	input() returns str. Convert before numeric calculations.

2 Operators at a glance

Purpose	Syntax or result	Remember
Assign or compare	`x = 3; x += 1` `x == 4  # True`	= assigns; == compares values.

Arithmetic	`7 / 2  # 3.5`
	`7 // 2  # 3`
	`-7 // 2 # -4`
		For int/float operands, / returns float; // floors, not truncates.

Remainder and power	`7 % 2  # 1`
	`2 ** 3  # 8`
	`7.0 // 2 # 3.0`
		// can return float. ** means exponentiation, not ^.

Compare	`==  !=  <  <=  >  >=` `0 <= x < 10`	Comparisons can be chained; all must hold.
Boolean logic	`and  or  not` `0 or 5  # 5`	and/or short-circuit and return an operand; not returns bool.
Identity	`a is b` `x is None`	Same object, not merely equal values. Prefer is None.
Membership	`"py" in "python" # True` `3 not in [1, 2] # True`	Checks membership; dictionaries check keys.

3 Precedence from high to low

Priority	Operators	Evaluation order
1	`(...)`	Grouped expression first
2	`**`	Right to left
3	`Unary +x, -x`	Signs; permitted directly in an exponent
4	`*  /  //  %`	Equal priority — left to right
5	`+  -`	Equal priority — left to right
6	`<  <=  >  >=  ==  !=` `in  not in  is  is not`	Equal priority; comparisons can be chained
7	`not`	Before and and or
8	`and`	Before or; short-circuits
9	`or`	Last; short-circuits

Sequences and flow control

4 Indexing and slicing

```
values = [10, 20, 30, 40]
values[0]       # 10
values[-1]      # 40
values[1:3]     # [20, 30] — stop excluded
values[:2]      # [10, 20]
values[::2]     # [10, 30]
values[::-1]    # [40, 30, 20, 10]
```

Syntax: sequence[start:stop:step]. Out-of-range indexing raises IndexError; slices clip to available bounds. A zero step raises ValueError. Strings also support indexing and slicing.

5 List methods and shared references

Operation	Example	Effect or return
Append	`items.append(4)`	Adds one element; mutates; returns None.
Remove	`items.remove(4)`	Removes first equal value; returns None; missing → ValueError.
Sort	`items.sort()`	Sorts in place; returns None. Do not assign its result to items.
Copy by slicing	`other = items[:]`	New shallow list; nested mutable objects are still shared.

Supplement	`items.extend([4, 5])`
	`items.pop()`
	`sorted(items)`
		extend adds each item, returns None; pop returns removed item; sorted returns new list.

```
a = [1, 2]
b = a               # Alias: same list
b.append(3)
print(a)            # [1, 2, 3]
c = a[:]            # Separate outer list
print(a == c, a is c) # True False
```

6 Decisions

```
sales = 300000
if sales >= 500000:
    print("Gold")
elif sales >= 300000:
    print("Silver")  # This branch runs
else:
    print("Standard")
```

Only the first matching branch runs. Use a colon and consistent indentation, normally 4 spaces. For overlapping >= thresholds, check highest first; for <= thresholds, lowest first. Nested if statements belong inside the selected branch.

7 Loops

```
total = 0
for number in range(1, 4):  # 1, 2, 3; stop excluded
    total += number
print(total)                # 6

count = 3
while count > 0:
    count -= 1              # Ensure a way to finish
```

Keyword	Meaning
break	Exit the innermost loop immediately.
continue	Skip the rest of this iteration; advance/recheck the loop.
pass	Do nothing; placeholder for a required statement.
range	range(stop) or range(start, stop, step); step cannot be 0.

Supplement: range(5, 0, -2) gives 5, 3, 1. A nested loop runs its inner loop for each outer item. A while loop can also end via break; a condition update is not always necessary.

Input output and functions

8 Console input and formatting

```
quantity = int(input("Quantity: "))  # Invalid integer → ValueError
price = 3.95
print("Price:", price)               # Price: 3.95
print("Price: " + str(price))        # Price: 3.95
print(f"Total: {price * quantity:.2f}")
print("Price: {:.2f}".format(price)) # Price: 3.95
print("A", "B", sep="-", end="!")   # A-B! (no newline)
```

input() strips the ending newline and returns text. "3" + "2" gives "32" and "3" * 2 gives "33"; these are string operations, not numeric arithmetic.

9 Files

Mode or method	Behaviour
`"r"`	Read text; default mode. Missing file → FileNotFoundError.
`"w"`	Write text; creates file or truncates existing content immediately.
`"a"`	Append text; creates file if missing; preserves existing content.
`read() / read(n)`	Read remaining content / up to n characters from current position.
`write(text)`	Write a string; returns number of characters written. No automatic newline.
`close() / with`	close() closes manually; with closes automatically on block exit.

```
with open("data.txt", "w", encoding="utf-8") as f:
    f.write("Hello\n")
with open("data.txt", "a", encoding="utf-8") as f:
    f.write("Again\n")
with open("data.txt", "r", encoding="utf-8") as f:
    text = f.read()
# text == "Hello\nAgain\n"
```

Manual equivalent: f = open(...), then use f, then f.close(). Prefer with so errors do not skip cleanup. File paths are relative to the current working directory unless absolute.

```
import os
exists = os.path.exists("data.txt")  # True for a file OR directory
# os.remove("data.txt")             # Deletes a file; may raise OSError
```

10 Command line arguments

```
import sys
# Run: python script.py Alice
script_name = sys.argv[0]           # Script name/path
if len(sys.argv) > 1:
    name = sys.argv[1]              # "Alice"; arguments are strings
```

sys.argv[2] is the second user argument. Check length before indexing; missing arguments otherwise raise IndexError.

11 Functions and documentation

```
def calculate_total(price, quantity=1):
    """Return the total price."""
    return price * quantity

calculate_total(3.95, 5)             # 19.75
calculate_total(3.95)                # 3.95
calculate_total(quantity=2, price=3) # 6
```

return ends the function and gives a value to the caller. No return, or bare return, yields None. print displays output; it does not replace return. Parameters are local names. Default values are evaluated once at definition; avoid mutable defaults such as [].

Use # for comments outside string literals. A docstring is the first statement in a module, class or function; triple quotes are conventional. help(calculate_total) displays it. Terminal command: python -m pydoc module_name.

Errors exceptions and testing

12 Identify the error

Category	Typical example	What to check
Syntax error	`if x > 0  # missing :`	Invalid syntax prevents that code from being parsed.
Runtime exception	`10 / 0`	An error during execution; can be handled with except.
Logic error	`Using > instead of >=`	Wrong behaviour without necessarily raising an exception.

Exception	Common cause
`NameError`	Undefined name; spelling/capitalisation mismatch.
`TypeError`	Unsupported operation or wrong argument type, e.g. "3" + 2.
`ValueError`	Acceptable type, invalid value, e.g. int("hello").
`IndexError`	Sequence index out of range.
`ZeroDivisionError`	Division or modulo by zero.
`FileNotFoundError`	Reading a path that does not exist.

13 Exception handling pattern

```
def divide_values(numerator, denominator):
    try:
        if denominator < 0:
            raise ValueError("Use a non-negative denominator")
        result = numerator / denominator
    except ZeroDivisionError:
        print("Cannot divide by zero")
    except ValueError as error:
        print(error)
    else:
        print(result)
    finally:
        print("Finished")

divide_values(10, 2)   # 5.0, then Finished
divide_values(10, 0)   # Cannot divide by zero, then Finished
```

try runs the risky code. except handles matching errors; other errors propagate. else runs after normal completion of try, not after an exception or an early return/break/continue. finally runs as control leaves the construct, including during return or exception propagation; abrupt process termination can bypass it.

raise triggers an exception. Negative-denominator rejection above is an example validation rule, not a Python restriction. Catch specific exceptions; keep the try block focused.

14 Minimal unittest pattern

```
import unittest

def multiply(a, b):
    return a * b

class TestMath(unittest.TestCase):
    def test_multiply(self):
        self.assertEqual(multiply(3, 4), 12)
        self.assertEqual(multiply(5, 0), 0)

if __name__ == "__main__":
    unittest.main()
```

Save as test_math.py; run python -m unittest test_math -v. Test method names start with test_. Two assertions in one method are one test, not two tests. The main guard runs the runner only when this file is executed directly.

Supplement: assertTrue(condition), assertFalse(condition), assertIsNone(value), assertAlmostEqual(actual, expected), and with self.assertRaises(ValueError): ... . Check a normal input, a boundary and an invalid input where relevant.

Modules and final exam checks

15 Standard library lookup

Module	Import and example	Result or purpose

math	`import math`
	`math.sqrt(16)`
	`math.floor(-3.2)`
	`math.ceil(-3.2)`
		4.0
		-4 (towards –infinity)
		-3 (towards +infinity)

datetime	`from datetime import datetime`
	`now = datetime.now()`
	`now.strftime("%Y-%m-%d")`
		Current local date and time; format as year-month-day.

datetime parsing	`datetime.strptime(`
	`  "2026-10-13", "%Y-%m-%d"`
	`)`
		Parse text to datetime; format must match the input.

random	`import random`
	`random.randint(1, 10)`
	`random.choice(["A", "B"])`
		Integer 1 through 10 inclusive; choose one item from a non-empty sequence.

sys	`import sys`
	`sys.argv`
	`sys.exit(0)`
		Argument list; exit with success status (raises SystemExit).

os	`import os`
	`os.path.exists("data.txt")`
	`# os.remove("data.txt")`
		Check path existence; remove deletes a file and may fail.

io	`import io`
	`f = io.StringIO("abc")`
	`f.read()  # "abc"`
		In-memory text stream. io.BytesIO(b"abc") is a binary stream.

Import forms: import math → math.sqrt(9); from math import sqrt → sqrt(9). Both give 3.0. Avoid naming your script math.py, random.py or unittest.py.

16 Common exam traps

Check	Correct interpretation
Names and types	Python is dynamically, not loosely, typed. first_name is the usual style; First_Name is a different name, not necessarily a different object.
Rounding	int(-3.9) → -3; -7 // 2 → -4. Floor division and truncation differ for negatives.
Identity	== compares values; is compares object identity. Reassignment does not guarantee a different object identity.
Boolean operators	and/or return operands and short-circuit. bool("False") is True because the string is non-empty.
Boundaries	Slice/range stop is excluded. randint(a, b) includes both endpoints.
Mutation	list.sort(), append(), remove() return None. b = a aliases; a[:] makes a shallow copy.
Control flow	Only the first matching if/elif branch runs. break exits a loop; continue skips one iteration; pass does nothing.
Input and output	input gives str; write does not add a newline; "w" destroys existing file contents.
Exceptions	Match the exception type. finally is cleanup, not a guarantee against process termination.
Code tracing	Track variable values, types, indentation, loop counts and exact output after each statement.